Foundations of Numerical Methods and Floating-Point Errors

1. Introduction

This project investigates floating-point representation, numerical error measurement, and the convergence of Taylor series approximations using Python. The objective is to understand how computers represent real numbers, how errors arise during calculations, and how increasing the number of terms improves the accuracy of a numerical approximation. NumPy was used for numerical calculations, while Matplotlib was used to visualize approximation errors.

2. Methodology

The experiments were carried out using Python in four scripts, with each script focusing on a different part of floating-point errors and numerical approximation.

First, floating-point precision was investigated. The numbers 0.4 and 0.3 were subtracted, and the result was compared with the expected answer of 0.1. The difference between the calculated and expected answers was checked to see whether the computer produced a small rounding error. Machine epsilon was also examined to understand the smallest difference that can be represented between floating-point numbers.

Second, errors were calculated. The difference between the calculated answer and the expected answer was used to find the absolute error. The relative error was then calculated to show the size of the error compared with the expected answer. The relative error was also converted into a percentage to make it easier to understand.

Third, the Taylor series method was used to approximate the value of (e^{0.4}). The calculation started with 1, and additional values were calculated and added one at a time using the Taylor series formula. The final approximation was compared with the reference value calculated using Python's numpy.exp(0.4) function. Absolute and relative errors were then calculated to measure the accuracy of the approximation.

Lastly, the effect of increasing the number of terms was investigated. The Taylor series calculation was repeated using between 1 and 10 terms. The absolute and relative errors were recorded at each step. Matplotlib was then used to draw graphs showing how the errors changed as more terms were added. This made it easier to observe how the accuracy of the approximation improved with additional terms.

3. Results and Observations

3.1 Floating-Point Representation

The first script investigated how Python handles decimal numbers. When 0.4 and 0.3 were subtracted, the result was compared with the expected answer of 0.1. The script also checked whether the two results were exactly equal and calculated the difference between them.

Machine epsilon was found to be approximately (2.22 \times 10^{-16}). This showed that computers have limits when representing numbers. Some decimal values cannot be stored exactly in binary form, which can lead to small rounding errors.

3.2 Absolute, Relative, and Percentage Errors

The second script calculated the absolute error, relative error, and percentage error using the result of subtracting 0.3 from 0.4.

The absolute error showed the difference between the calculated result and the expected answer of 0.1. The relative error showed this difference in relation to the expected answer, while the percentage error expressed it as a percentage. These calculations helped demonstrate different ways of measuring the accuracy of a result.

3.3 Taylor Series Approximation

The third script used the Taylor series method to estimate the value of (e^{0.4}). The reference value was calculated using Python's numpy.exp(0.4) function and was approximately 1.491824697641.

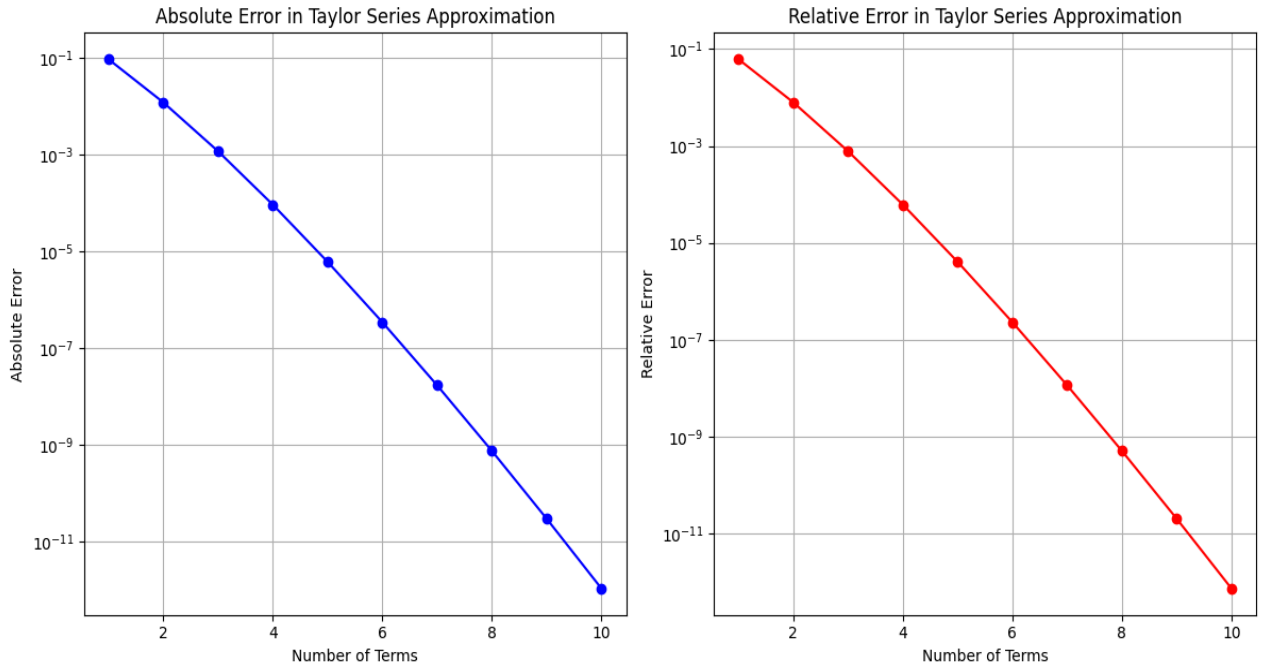
After adding 10 terms, the Taylor series approximation was 1.491824697640. This value was very close to the reference value. The absolute error was approximately (1.0871 \times 10^{-12}), while the relative error was approximately (7.2873 \times 10^{-13}).

These results show that the Taylor series method produced a highly accurate approximation using 10 terms. The very small errors indicate that the calculated value was almost identical to the reference value.

3.4 Error Visualization

The fourth script repeated the Taylor series calculations using between 1 and 10 terms. At each step, the absolute and relative errors were calculated and stored.

Matplotlib was then used to draw two graphs showing how the errors changed as the number of terms increased. The graphs showed that both types of error generally decreased as more terms were added. This made it easier to see how the accuracy of the approximation improved, as shown in the below line graphs.



Overall, the four scripts demonstrated how rounding errors occur, how errors can be measured, how the Taylor series method improves an approximation, and how graphs can help us understand changes in numerical accuracy.

4. Conclusion

The experiments showed that computers can produce small rounding errors because they cannot represent every decimal number exactly. The error calculations demonstrated how the accuracy of a result can be measured in different ways. The Taylor series experiment showed that adding more terms generally improves an approximation and reduces its error. Finally, the graphs provided a clear way to observe this improvement. The project helped demonstrate the importance of numerical precision and error measurement when performing mathematical calculations using Python.